

Shiraz University

Department of Computer Science & Engineering

Text Mining & Natural Language Processing

Assignment 3

Aspect-Based Hybrid Recommender Systems via Rule-Based Text Mining on Yelp Reviews

Instructor: Dr. Mostafa Fakhrahmad

Teaching Assistants: Alireza Shayan, Hossein Karami

Course: Text Mining & NLP

Term: Spring 1405 / 2026

Start: **Friday, July 24, 2026 / 2 Mordad 1405**

Deadline: **23:59 of Thursday, August 13, 2026 / 22 Mordad 1405**

Contents

1 Overview & Learning Goals **2**
1.1 Core Objective . . . 2
1.2 Learning Goals . . . 2

2 Dataset Acquisition & File Structure **3**
2.1 Dataset Download & Source . . . 3
2.2 Directory Layout & Filtered Subsets . . . 3
2.3 Data Fields & Schema . . . 4
2.4 Data Usage Policy . . . 4

3 Detailed Task Breakdown **5**
3.1 Raw Dataset Parsing & Deterministic Filtering (NLP Phase 0) . . . 5
3.2 Semantic Aspect Extraction and Canonical Mapping (NLP Phase 1) . . . 6
3.3 Dependency-Guided Aspect-Level Sentiment Scoring (NLP Phase 2) 7
3.4 Hybrid Scoring & Matrix Factorization . . . 8
3.5 Top-K Recommendation & Abstention . . . 9
3.6 Evaluation Metrics . . . 10
3.7 Bonus Task: Word2Vec Semantic Aspect Expansion (+15% Bonus) . . . 11
3.8 Bonus Task: Transformer-Based NLP Extension (+30% Bonus) . . . 11
3.9 Baselines to Implement . . . 12

4 Conceptual & Mathematical Deep Dives **13**

5 Standard CLI Interface & Code Structure **13**
5.1 Command Line Interface . . . 13
5.2 Recommended Code Structure . . . 14

6 Required Output Files & Trace Logging **15**
6.1 Output Files Schema . . . 15

7 Report Requirements **16**

8 Grading **18**

9 Submission **19**

10 Key Hints **19**

11 Suggested References **19**

1. Overview & Learning Goals

1.1. Core Objective

In standard collaborative filtering, recommender systems rely strictly on numerical interaction matrices (e.g., star ratings). However, ratings alone are inherently opaque: two users might both rate a restaurant $3/5$ stars, but for entirely opposite reasons (e.g., one loved the food but hated the noise; the other loved the quiet ambiance but found the food bland).

In this assignment, you will build an **Aspect-Based Hybrid Recommender System (AB-RecSys)** that mines textual customer reviews to unlock fine-grained user preferences and business attributes. The core mandatory pipeline must operate under strict computational constraints: it must execute efficiently on standard multi-core CPUs using rule-based NLP (`VADER`) **without requiring dedicated GPU acceleration or large language model (LLM) fine-tuning**. However, as an optional bonus task, students may implement Transformer-based sentiment modeling alongside VADER for up to +30% bonus credit.

Your system must perform three core tasks:

- **Aspect Extraction & Sentiment Analysis:** Map review mentions to the five fixed domain aspects `food`, `service`, `ambiance`, `price`, and `location`, then estimate aspect-level sentiment using classical NLP.
- **Hybrid Scoring:** Combine explicit ratings (1–5 stars) with extracted aspect-sentiment alignment scores using matrix decomposition and vector space models.
- **Top-K Recommendation & Abstention:** Produce personalized Top-K recommendations while gracefully handling cold-start or low-confidence scenarios.

1.2. Learning Goals

By completing this assignment, students will:

1. Process real-world, large-scale unstructured text corpora efficiently using batch processing and multi-processing in `spaCy`.
2. Extract fine-grained opinion targets (Aspect-Based Sentiment Analysis) using Part-of-Speech (POS) tagging and rule-based sentiment lexicons (`VADER`) for the mandatory baseline, alongside optional Transformer-based models (`BERT`, `RoBERTa`, Hugging Face) for +30% bonus credit.
3. Construct sparse user–item matrices and five-column user-aspect and item-aspect profile matrices using `scipy.sparse`.
4. Implement Collaborative Filtering (SVD Matrix Factorization) from scratch or using lightweight numerical libraries.
5. Combine collaborative and textual content signals into a unified hybrid scoring

framework.

6. Evaluate recommender systems using ranking metrics (HR@K, NDCG@K) alongside error metrics (RMSE).
7. Conduct detailed ablation studies and qualitative error analyses.

2. Dataset Acquisition & File Structure

2.1. Dataset Download & Source

A fixed Yelp Open Dataset snapshot and its SHA-256 checksum are provided through the course LMS. This course snapshot is the *only* permitted input release. It is derived from the official Yelp Open Dataset, but students must not substitute a newer download because the upstream files, coverage, and sizes can change. The official landing page is:

`https://business.yelp.com/external-assets/files/Yelp-JSON.zip`

Verify the supplied archive with `sha256sum` before extraction. Extract it into your working directory. The required raw files are:

- `Dataset_User_Agreement.pdf` (78 KB)
- `yelp_academic_dataset_business.json` (113 MB): Metadata regarding businesses (ID, name, categories, location, star rating, total reviews).
- `yelp_academic_dataset_checkin.json` (274 MB): Business check-ins.
- `yelp_academic_dataset_review.json` (5.0 GB): Full customer reviews, user IDs, ratings, dates, and unstructured review text.
- `yelp_academic_dataset_tip.json` (172 MB): Tips written by users.
- `yelp_academic_dataset_user.json`: User metadata (not used by this assignment).

2.2. Directory Layout & Filtered Subsets

To ensure tractable CPU execution and reproducibility, pre-filter the supplied raw archive for one metropolitan region. Phase 0 deterministically creates `businesses.jsonl`, `train_reviews.csv`, `dev_reviews.csv`, `dev_qrels.tsv`, and `dev_users.csv`. The file `test_users.csv` is provided separately with the assignment dataset; its associated future interactions are not distributed. The workspace must follow this structure:

```
HW3_data/
|-- Yelp_JSON.zip        <- Raw downloaded dataset archive from
   Yelp direct link
|-- Yelp_JSON/           <- Extracted directory containing raw
   Yelp JSON files
|-- businesses.jsonl     <- Metadata for businesses (business_id,
```

```
    name, categories, city, stars, review_count)
|-- train_reviews.csv    <- Training interactions with full
   review text
|-- dev_reviews.csv      <- Public temporal validation
   interactions
|-- dev_qrels.tsv        <- Ground-truth positive interactions
   for dev evaluation
|-- dev_users.csv        <- Evaluation users generated in Phase 0
|-- test_users.csv       <- Course-supplied target user IDs for
   blind evaluation
+-- README_dataset.txt   <- Detailed schema and subset
   description
```

Listing 1 – *Dataset directory layout*

All text files are UTF-8 encoded.

2.3. Data Fields & Schema

- `businesses.jsonl`: Contains `business_id`, `name`, `categories`, `city`, `stars`, and `review_count`.
- `train_reviews.csv`: Contains `review_id`, `user_id`, `business_id`, `stars` (1.0–5.0), `date`, and `text` (unstructured review text).
- `dev_reviews.csv`: Has exactly the same CSV columns and CSV dialect as `train_reviews.csv`. CSV files have a header, UTF-8 encoding, RFC 4180 quoting, and dates formatted as `YYYY-MM-DD`.
- `dev_qrels.tsv`: Has a header and columns `user_id`, `business_id`, and `relevance`; duplicate user–business pairs are collapsed using the maximum relevance label.
- `dev_users.csv`: Contains, under header `user_id`, the sorted IDs of dev users with at least three training reviews and at least one relevant dev business.
- `test_users.csv`: Is course-supplied and contains only `user_id`. It is the target list for blind evaluation; neither its qrels nor its held-out reviews are provided.

2.4. Data Usage Policy

- `train_reviews.csv` must be used to construct the vocabulary, extract aspects, compute sentiment scores, and train the matrix factorization model.
- `dev_reviews.csv`, `dev_qrels.tsv`, and `dev_users.csv` are reserved for hyperparameter selection (α and confidence threshold τ) and local validation. They must never be used to fit model parameters, construct aspect profiles, or compute popularity.
- `test_users.csv` contains target user IDs for blind evaluation. Generated recommendations (`test_recommendations.csv`) will be evaluated against hidden ground-truth interactions. No test interaction data may be used during training

or tuning.

3. Detailed Task Breakdown

3.1. Raw Dataset Parsing & Deterministic Filtering (NLP Phase 0)

To ensure full reproducibility and standardized evaluation, implement a deterministic preprocessing module (Phase 0) that converts the supplied raw JSON files into the required workspace structure. Read JSON Lines one record at a time; do not load an entire raw file into memory. If a required field is missing, null, malformed, or has an invalid date/rating, discard that record and report its count in `summary.json`. You must follow these rules exactly:

1. **Business Filtering:** Retain a business when `city` is exactly `"Philadelphia"` and the comma-separated `categories` string, after splitting on commas and stripping whitespace, contains the exact category `"Restaurants"` or `"Food"`. Treat a null category as no categories. Sort retained records by `business_id` and write only `business_id`, `name`, `categories`, `city`, `stars`, and `review_count` to `businesses.jsonl`.
2. **Review Filtering & Sorting:** Retain reviews for retained business IDs. Parse `date` as an ISO calendar date and sort retained reviews by the tuple `(date, review_id)` in ascending lexicographic order. A `review_id` must occur at most once; retain the first record in that order if a duplicate is encountered.
3. **Train/Dev Temporal Split:**
   - Reviews with `date` strictly prior to `2021-01-01` must be written to `train_reviews.csv`.
   - Reviews with `date` on or after `2021-01-01` must be written to `dev_reviews.csv`.
4. **Ground-Truth Qrels Generation (`dev_qrels.tsv`):** For each distinct (u, i) pair in `dev_reviews.csv`, assign relevance `1` if any of its dev reviews has `stars` ≥ 4.0, and `0` otherwise. Write one row per pair, sorted by `(user_id, business_id)`, with the required header and tab separators.
5. **Dev-user Selection (`dev_users.csv`):** Extract each distinct `user_id` that has at least three retained training-review rows and at least one relevance-`1` pair in `dev_qrels.tsv` whose business is present in the training catalogue (has at least one training review). Sort IDs lexicographically and write them under header `user_id`. Do *not* create or overwrite the course-provided `test_users.csv`.

3.2. Semantic Aspect Extraction and Canonical Mapping (NLP Phase 1)

Instead of using noisy raw nouns as distinct aspects, you are required to implement a semantic mapping step that maps arbitrary terms to a set of canonical aspects.

1. **Tokenization & POS Tagging:** Use `spacy==3.7.5` and `en_core_web_md==3.7.1`. Process review texts with parser, tagger, and lemmatizer enabled. For each review, enumerate noun chunks in document order. A chunk is one candidate mention, represented by its syntactic root token (`token = span.root`); never emit an additional unigram for a token belonging to any noun chunk. Ignore chunks whose root is a pronoun, stop word, punctuation, or has no vector.
2. **Semantic Aspect Mapping:** The five canonical aspects are fixed: `food`, `service`, `ambiance`, `price`, and `location`. Map each candidate root token (`token`) to the seed token with highest cosine similarity between their spaCy vectors (`token.similarity(seed`. Accept it only if the maximum similarity is at least $\theta = 0.35$. Resolve an exact tie by the listed seed order. Do not use external lexicons, embeddings, LLMs, manually added mappings, or dev/test text for this core mapping. The profile matrices therefore always have exactly five columns; no aspect-count ablation is permitted.

   ▹ **Note**

   spaCy Model & API Requirements: To compute vector similarity, you must use a `spaCy` model with static word vectors (e.g., `en_core_web_md` or `en_core_web_lg`); the default lightweight model (`en_core_web_sm`) lacks static word vectors and will produce invalid similarity scores. To ensure consistent vector similarity scores across all submissions, vector similarity must be computed directly between single tokens (`token.similarity(seed_token)` where `token = span.root`). Calling `span.similarity()` on multi-word `Span` objects generates phrase-level average vectors and is not permitted for this step. Furthermore, dependency parsing (`dep_`, `head`) operates on individual `Tokens` (e.g., the root noun of a chunk).

3. **Multiprocessing Requirement:** Text parsing **must** use `nlp.pipe(..., n_process=K, batch_size=1000)`, where K is the positive integer passed through `--n-process`. Preserve input order when pairing documents with review metadata.

3.3. Dependency-Guided Aspect-Level Sentiment Scoring (NLP Phase 2)

To perform precise aspect-based sentiment analysis, you must not score the entire sentence. Implement the following dependency-guided extraction engine:

1. **Syntactic Dependency Parsing:** For each matched aspect term in review r, use spaCy's dependency parser to traverse the syntactic tree.
2. **Opinion Modifier Extraction:** Let the root token of a matched chunk be the aspect head. Extract (i) each adjectival child of that head with dependency `amod`, and (ii) each adjectival complement of a copular verb for which the head is an `nsubj` or `nsubjpass`. These are the only required patterns.
3. **Opinion Phrase Construction and Negation:** For every extracted adjective, include its contiguous adverbial-modifier children (`advmod`) and negation children (`neg`); sort selected tokens by token index and join their surface forms with spaces. Thus "extremely clean" and "not good" are valid phrases. Do not negate a phrase merely because a negator appears elsewhere in the sentence.
4. **Lexical Sentiment Scoring:** Score each phrase with the standard, unmodified NLTK VADER `SentimentIntensityAnalyzer` and use its `compound` value. For one review r and canonical aspect a, let $s(r, a)$ be the arithmetic mean of all phrases from all mentions mapped to a; if there are mapped mentions but no extracted phrase, set $s(r, a) = 0.0$. A review with no mapped mention of a has no $s(r, a)$ value and is excluded from $\mathcal{R}_i^{(a)}$.
5. **Profile Aggregation:** Aggregate aspect mentions and sentiments to build two profile matrices:
   - **User-Aspect Attention/Weight Matrix ($\mathbf{U} \in \mathbb{R}_{\geq 0}^{|U| \times 5}$):** Non-negative weights representing the degree of attention or importance user u places on aspect a. This can be calculated as the normalized mention frequency of aspect a across user u's reviews:

$$\mathbf{U}_{u,a} = \frac{\text{count}(u, a)}{\max\left(\sum_{a'=1}^{5} \text{count}(u, a'),\ \epsilon\right)} \quad \in [0, 1] \tag{1}$$

     where $\text{count}(u, a)$ is the total number of mapped chunk mentions of aspect a across user u's training reviews, and $\epsilon = 10^{-9}$. Note that $\sum_{a=1}^{5} \mathbf{U}_{u,a} = 1$ whenever the user mentions at least one aspect; otherwise its row is all zeros.
   - **Item-Aspect Quality/Sentiment Matrix ($\mathbf{V} \in \mathbb{R}^{|I| \times 5}$):** Average sentiment score received by item i for aspect a. Let $\mathcal{R}_i^{(a)} \subseteq \mathcal{R}_i$ be the subset of reviews of item i that **mention** aspect a (regardless of whether an opinion modifier was found), and let $\text{count}(i, a) = |\mathcal{R}_i^{(a)}|$:

$$\mathbf{V}_{i,a} = \frac{\sum_{r \in \mathcal{R}_i^{(a)}} s(r, a)}{\max\left(\text{count}(i, a),\ \epsilon\right)} \quad \in [-1, +1] \tag{2}$$

where $s(r, a) \in [-1, +1]$ is the VADER compound score of the extracted opinion phrase for aspect a in review r (set to 0.0 if no modifier was found; see Phase 2, Step 4).

3.4. Hybrid Scoring & Matrix Factorization

The system must compute a composite prediction score $\hat{R}_{u,i}$ for user u and business i:

$$\hat{R}_{u,i} = \alpha \cdot \text{Score}_{\text{CF_norm}}(u, i) + (1 - \alpha) \cdot \text{Score}_{\text{Aspect_norm}}(u, i) \tag{3}$$

1. Collaborative Filtering Component (Score_{CF})

Train a classical Matrix Factorization model (e.g., SVD or Biased MF) on the explicit rating matrix $\mathbf{R} \in \mathbb{R}^{|U| \times |I|}$:

$$\text{Score}_{\text{CF}}(u, i) = \mu + b_u + b_i + \mathbf{p}_u^T \mathbf{q}_i \tag{4}$$

where μ is the global average rating, b_u, b_i are user/item biases, and $\mathbf{p}_u, \mathbf{q}_i$ are k-dimensional latent vectors. Train this model only on training-review rows. If multiple training reviews share a (u, i) pair, use their arithmetic mean as the single observed rating. Fix all pseudo-random sources with `--seed`; report the implementation, latent dimension, optimiser, learning rate, regularization, epochs, and seed in `summary.json`.

2. Aspect Sentiment Alignment Component ($\text{Score}_{\text{Aspect}}$)

To align non-negative user aspect attention weights $\mathbf{U}_u \in [0, 1]^5$ with item aspect quality ratings $\mathbf{V}_i \in [-1, +1]^5$, compute the weighted aspect alignment score:

$$\text{Score}_{\text{Aspect}}(u, i) = \sum_{a=1}^{5} \mathbf{U}_{u,a} \cdot \mathbf{V}_{i,a} \quad \in [-1, +1] \tag{5}$$

▹ **Note**

Why Non-Negative User Weights? If user profiles contained signed sentiment, taking a dot product between a negative user sentiment (e.g., user complained about bad service, -0.8) and a negative item sentiment (item has terrible service, -0.8) would yield a positive product ($+0.64$), falsely recommending an item with poor service to a sensitive user! Using non-negative user weights $\mathbf{U}_{u,a} \geq 0$ ensures that bad item performance on highly prioritized aspects strictly lowers the alignment score.

3. Global Min-Max Normalization

To prevent scale mismatch (where Score_{CF} operates on a 1–5 star range, and $\text{Score}_{\text{Aspect}}$ operates on a $[-1, +1]$ range), you must scale both components to $[0, 1]$ globally before blending:

$$\text{Score}_{\text{CF_norm}}(u, i) = \frac{\text{clip}\big(\text{Score}_{\text{CF}}(u, i),\ 1.0,\ 5.0\big) - 1.0}{4.0} \tag{6}$$

$$\text{Score}_{\text{Aspect_norm}}(u, i) = \frac{\text{Score}_{\text{Aspect}}(u, i) + 1.0}{2.0} \tag{7}$$

The $\text{clip}(\cdot, 1.0, 5.0)$ operation is necessary because SVD with bias terms can produce predicted ratings outside the $[1.0, 5.0]$ star range; clipping ensures the normalized score remains in $[0, 1]$. If a user has no mapped training aspects, define $\text{Score}_{\text{Aspect}}(u, i) = 0$ for every item, so its normalized component is 0.5; the abstention rule may then select the popularity fallback.

4. Rating Rescaling for Evaluation

For evaluating rating prediction accuracy (RMSE) against ground-truth star ratings from `dev_reviews.csv`, rescale the blended score $\hat{R}_{u,i} \in [0, 1]$ back to the star rating scale:

$$\hat{R}^{\text{stars}}_{u,i} = 1.0 + 4.0 \cdot \hat{R}_{u,i} \tag{8}$$

3.5. Top-K Recommendation & Abstention

Define the candidate catalogue as businesses with at least one training review. For a user u, candidates are catalogue businesses that u did not review in training. The same procedure is used for dev recommendations, with `dev_users.csv`, and for blind recommendations, with `test_users.csv`. For each target user:

1. Rank candidates by descending score; break exact numeric ties by ascending `business_id`. Generate the first $\min(10, |C_u|)$ items. A target user with no candidates must have zero output rows and an explicit trace entry.
2. **Abstention Rule:** If a user has fewer than $M = 5$ training-review rows, or if candidate set C_u is non-empty and $\max_{i \in C_u} \hat{R}_{u,i} < \tau$ with $\tau = 0.30$, mark the user `low_confidence` and rank candidates by descending training-review count instead. Break popularity ties by ascending `business_id`. For fallback rows, write popularity divided by the maximum training-review count in the catalogue as `score` and `is_fallback=true`; otherwise write $\hat{R}_{u,i}$ and `is_fallback=false`. The count threshold and τ may be tuned on dev only.

3.6. Evaluation Metrics

The following metrics must be computed on the public dev set using `dev_users.csv` as U_{eval}; the same definitions apply to hidden test qrels. Ranking is a batch temporal holdout: dev interactions are never training observations, but all catalogue items not seen in training remain candidates. In every metric, use only user–business pairs whose business is in the training catalogue.

- **RMSE (Root Mean Squared Error):** Measures rating prediction accuracy against ground-truth star ratings from `dev_reviews.csv`:

$$\text{RMSE} = \sqrt{\frac{1}{|\mathcal{D}|} \sum_{(u,i)\in\mathcal{D}} \left(\hat{R}^{\text{stars}}_{u,i} - R^{\text{stars}}_{u,i}\right)^2} \tag{9}$$

  where $\mathcal{D}$ contains one pair for each distinct (u, i) dev pair for users in U_{eval}, with R^{stars} equal to the mean of that pair's dev star ratings. $\hat{R}^{\text{stars}}$ is the rescaled predicted rating. Fallback recommendations do not alter RMSE: always use the hybrid rating prediction for this metric.
- **HR@K (Hit Rate at K):** Fraction of evaluated users for whom at least one relevant item (relevance = 1 in `dev_qrels.tsv`) appears in their top-K recommendations:

$$\text{HR@}K = \frac{\left|\{u \in U_{\text{eval}} : \text{Top-}K(u) \cap \text{Rel}(u) \neq \emptyset\}\right|}{|U_{\text{eval}}|} \tag{10}$$

  where $\text{Rel}(u)$ is the set of relevance-1 candidate items for user u. The construction of `dev_users.csv` guarantees this set is nonempty.
- **NDCG@K (Normalized Discounted Cumulative Gain at K):** Measures ranking quality with position-weighted binary gains:

$$\text{NDCG@}K = \frac{1}{|U_{\text{eval}}|} \sum_{u\in U_{\text{eval}}} \frac{\text{DCG@}K(u)}{\text{IDCG@}K(u)}, \quad \text{DCG@}K(u) = \sum_{j=1}^{K} \frac{\text{rel}(u, \pi_j)}{\log_2(j+1)} \tag{11}$$

  where π_j is the item at rank j, $\text{rel}(u, i) \in \{0, 1\}$ is the binary relevance from `dev_qrels.tsv` (zero for an unlisted item), and $\text{IDCG@}K = \sum_{j=1}^{\min(K,|\text{Rel}(u)|)} 1/\log_2(j+1)$. This denominator is nonzero for every evaluated user (if for any edge case $\text{IDCG@}K(u) = 0$, define $\text{NDCG@}K(u) = 0.0$).

3.7. Bonus Task: Word2Vec Semantic Aspect Expansion (+15% Bonus)

In addition to the mandatory rule-based VADER pipeline and the optional Transformer extension, students can earn up to **+15% bonus credit** by implementing a custom **Word2Vec** semantic expansion model using `gensim`.

▹ **Note**

Bonus Objective: Standard spaCy vector mapping relies on static pre-trained vectors which might miss domain-specific slang or restaurant-specific terminology (e.g., mapping unique food items or colloquial service terms). Training a domain-specific Word2Vec model on the Yelp review corpus helps capture contextual semantic similarities natively.

To complete this bonus task, you must:

- **Corpus Preparation:** Tokenize and lemmatize the review texts from `train_reviews.csv` into sentences, removing stop words and punctuation.
- **Model Training:** Train a Word2Vec model (Continuous Bag of Words or Skip-gram) using `gensim.models.Word2Vec` with a recommended vector dimensionality of $d \in [50, 100]$, minimum word count frequency threshold of 5, and window size of 5. Fix the random seed using `--seed`.
- **Semantic Similarity Substitution:** Replace or augment the spaCy vector similarity lookup (Section 3.2) with your trained Word2Vec model's cosine similarity to map extracted noun chunks to the 5 canonical aspects (`food`, `service`, `ambiance`, `price`, and `location`).
- **Comparative Evaluation:** Evaluate and report the performance impact of the Word2Vec-enhanced mapping on the dev set (RMSE, HR@10, NDCG@10) compared against the standard spaCy baseline in Section 9 of your report.

3.8. Bonus Task: Transformer-Based NLP Extension (+30% Bonus)

In addition to the mandatory rule-based NLP pipeline, students can earn up to **+30% bonus credit** by designing and integrating a Transformer-based NLP extension alongside VADER.

▹ **Note**

Mandatory Requirement Reminder: Implementation of the dependency-guided VADER sentiment engine (NLP Phase 2) **remains strictly mandatory** for the core hybrid pipeline. The Transformer component serves as an optional bonus extension for comparative evaluation and sentiment refinement.

Students may choose one or a combination of the following Transformer-based extensions:

- **Transformer Sentiment & ABSA Scoring:** Use pre-trained or fine-tuned Trans-

former models (e.g., `distilbert-base-uncased-finetuned-sst-2-english`, `roberta-base`, or specialized ABSA checkpoints) via Hugging Face on CPU/GPU to score extracted opinion phrases or review sentences, comparing sentiment compound scores with VADER.

- **Zero-Shot / Few-Shot Aspect & Sentiment Classification:** Utilize Hugging Face `pipeline("zero-shot-classification")` or `pipeline("sentiment-analysis")` to perform aspect classification or context-aware sentiment analysis on complex or sarcastic review text.
- **Transformer Fine-Tuning on Yelp Reviews:** Fine-tune a lightweight Transformer model (e.g., DistilBERT or RoBERTa) on Yelp review rating-text pairs to produce calibrated aspect sentiment embeddings or domain-adapted sentiment predictors.
- **Comparative Performance & Latency Evaluation:** Evaluate the Transformer-enhanced hybrid model on the dev set (RMSE, HR@10, NDCG@10) against the mandatory VADER hybrid model, analyzing inference throughput and computational trade-offs.
- **Hybrid Creative Extensions:** Combine Transformer sentiment scores with advanced features such as recency-weighted aspect profiles, contrastive aspect explanations (e.g., explaining why Item A ranks above Item B based on aspect alignment), or corpus-based lexicon calibration.

The bonus extension must be toggleable via an optional CLI argument (e.g., `--use-transformer`) and documented thoroughly in Section 9 of the report (`report.pdf`).

3.9. Baselines to Implement

You must compare your proposed hybrid model against three mandatory baselines:

Table 1 – *Required baseline systems*

Baseline	Description
Popularity-Based	Rank items globally by total review count in the training set (`train_reviews.csv`). For RMSE evaluation, use the business's average star rating **computed from training reviews only** (not from `businesses.jsonl`, which may include dev-set data) as the predicted rating.
Ratings-Only SVD	Standard Collaborative Filtering SVD without textual signals ($\alpha = 1.0$). Applies the same abstention fallback rule for low-confidence or sparse users as the hybrid model.
Aspect Content-Only	Ranking purely using Aspect Alignment Score ($\alpha = 0.0$).

4. Conceptual & Mathematical Deep Dives

To demonstrate a rigorous theoretical understanding of the NLP and recommendation algorithms, you must address the following questions in your report (Item 11 of Section 7):

1. **Mathematical Properties of Global vs. Local Min-Max Normalization:** Let $S_{\text{CF}}(u, \cdot)$ be the set of Collaborative Filtering scores for user u, and $S_{\text{Aspect}}(u, \cdot)$ be the set of aspect alignment scores for user u. Suppose we normalize these scores *locally* (per user) to $[0, 1]$ instead of *globally*. Prove that if $S_{\text{CF}}(u, \cdot)$ has zero variance (e.g., SVD predicts the exact same score for all candidate items for user u), the local min-max normalization is undefined. Show how this affects the ranking stability and explains why global normalization is numerically superior.
2. **Review Bias and Sentiment Calibration:** Ratings and textual sentiment frequently exhibit mismatch (e.g., a user writes "the service was okay, but the pasta was legendary" and rates it $5/5$ stars). Analyze the distribution of user ratings versus VADER sentiment compound scores. Mathematically, how does this discrepancy affect the aspect alignment score? Propose a normalization or clipping strategy to calibrate rating and textual sentiment scales.
3. **Profile Matrix Sparsity Bound:** Suppose each user reviews at most $R_{\max}$ items, and each review mentions at most $A_{\max}$ unique aspects (where $A_{\max} \leq 5$). Derive a **lower bound** on the sparsity (percentage of zero entries) of the User-Aspect Preference Matrix $\mathbf{U} \in \mathbb{R}^{|U| \times 5}$ in terms of $R_{\max}$ and $A_{\max}$. Discuss separately the case when $R_{\max} \cdot A_{\max} < 5$ (where a non-trivial bound exists) and when $R_{\max} \cdot A_{\max} \geq 5$ (where the bound becomes trivial). Comment on the role of $|U|$ in your derivation.

5. Standard CLI Interface & Code Structure

5.1. Command Line Interface

The entire training, aspect extraction, and recommendation pipeline must execute via a single CLI invocation. When `--raw-dir` is specified, `system.py` must always execute the Phase 0 data pipeline to regenerate the filtered datasets (to ensure deterministic reproducibility verification). If `--raw-dir` is omitted, `system.py` will load the pre-filtered dataset files directly:

```
python system.py \
--raw-dir       HW3_data/Yelp_JSON \
--train-reviews HW3_data/train_reviews.csv \
```

```
--businesses    HW3_data/businesses.jsonl \
--dev-reviews   HW3_data/dev_reviews.csv   \
--dev-qrels     HW3_data/dev_qrels.tsv     \
--dev-users     HW3_data/dev_users.csv     \
--test-users    HW3_data/test_users.csv    \
--outdir        results
```

Listing 2 – *Main CLI invocation*

Recommended optional CLI arguments:

```
--alpha 0.7          # hybrid weight parameter (0.0 to 1.0)
--n-process 4        # CPU cores for spaCy parallel processing
--min-interactions 5 # minimum user interactions M for abstention
--threshold 0.3      # abstention confidence threshold (tau in
   [0, 1])
--seed 42            # random seed for reproducibility
--use-transformer    # optional flag to enable Transformer bonus
   model (+30% bonus)
```

Listing 3 – *Optional CLI arguments*

5.2. Recommended Code Structure

Modular organization is strongly encouraged:

```
project_root/
|-- system.py            <- CLI entry point and pipeline
   orchestrator
|-- text_processor.py    <- spaCy parallel parsing & aspect
   extraction
|-- sentiment.py         <- VADER aspect-level sentiment scoring
   (mandatory)
|-- transformer_model.py <- Transformer sentiment scoring & HF
   pipeline (optional bonus)
|-- bonus_notebook.ipynb <- optional Jupyter Notebook for Colab/
   bonus experiments
|-- hybrid_model.py      <- SVD training and hybrid score
   combination
|-- evaluator.py         <- metric computation (RMSE, HR@K,
   NDCG@K)
|-- config.json          <- hyperparameter configurations
+-- results/
|   |-- dev_recommendations.csv
|   |-- test_recommendations.csv
|   |-- dev_metrics.json
|   |-- summary.json
```

```
|   |-- trace.jsonl
|   +-- report.pdf
+-- extra/                <- optional plots and logs
```

Listing 4 – *Recommended project structure*

6. Required Output Files & Trace Logging

All generated output files must be placed inside the directory specified by `--outdir` (e.g., `results/`).

6.1. Output Files Schema

1. `dev_recommendations.csv` & `test_recommendations.csv`: Long-format recommendations for every user in `dev_users.csv` and `test_users.csv`, respectively. Each user has exactly 10 rows unless fewer than 10 candidates exist. Rows are ordered by `user_id`, then ascending `rank`; ranks begin at 1 and `business_id` values must be unique within a user. The exact header and lowercase Boolean literals are:

Listing 5 – *Required CSV schema for dev_recommendations.csv and test_recommendations.csv*

```
user_id,business_id,rank,score,
   is_fallback
u_10842,b_4011z,1,0.8123,false
u_10842,b_8820x,2,0.7950,false
u_55231,b_1234a,1,0.0412,true
...
```

   The `score` column contains the hybrid score $\hat{R}_{u,i}$ for normal recommendations, or the normalized popularity score for fallback recommendations. The `is_fallback` column indicates whether the recommendation was generated via the popularity fallback mechanism.
2. `dev_metrics.json`: A valid UTF-8 JSON object containing top-level keys for the three baselines and the mandatory hybrid model (e.g., `"Popularity-Based"`, `"Ratings-Only SVD"`, `"Aspect Content-Only"`, `"VADER-Hybrid"`). Students attempting bonus tasks should also include corresponding top-level keys (e.g., `"Word2Vec-Hybrid"` or `"Transformer-Hybrid"`) with their `rmse`, `hr@10`, and `ndcg@10` values to facilitate automated evaluation.
3. `summary.json`: A valid UTF-8 JSON object containing raw-record discard counts, training/dev row counts, total tokens processed, five aspect frequencies, wall-clock runtime in seconds, package/model versions, and all final hyperparameters.

4. `trace.jsonl`: One valid JSON object per line for the first 20 `test_users.csv` IDs in lexicographic order (or all IDs if fewer than 20), including the abstention decision. Do not include review text or other personally identifying data.

Listing 6 – *Sample entry in trace.jsonl*

```
{
        "user_id": "u_10842",
        "user_train_review_count": 8,
        "extracted_user_aspect_preferences": [
        {"aspect": "service", "score": 0.55},
        {"aspect": "food", "score": 0.25},
        {"aspect": "ambiance", "score": 0.05},
        {"aspect": "price", "score": 0.10},
        {"aspect": "location", "score": 0.05}
        ],
        "top_recommended_item": {
                "business_id": "b_4011z",
                "business_name": "Tacos & Cantina",
                "svd_score_norm": 0.78,
                "aspect_score_norm": 0.88,
                "final_hybrid_score": 0.81,
                "item_aspect_alignment": [
                {"aspect": "service", "item_sentiment": 0.91, "
                    contribution": "positive"},
                {"aspect": "price", "item_sentiment": -0.10, "
                    contribution": "neutral"}
                ]
        },
        "abstain_fallback_triggered": false,
        "decision_reason": "Hybrid score 0.81 exceeds threshold
            0.30; high aspect alignment on 'service'."
}
```

7. Report Requirements

The report (`report.pdf`) must cover the following 13 sections:

1. **Problem Setting:** Why ratings alone fail to capture user preferences. The importance of review text mining.
2. **Method Overview:** End-to-end architecture diagram from raw CSV/JSONL to final hybrid recommendations.
3. **Aspect Extraction & Semantic Mapping:** Detail POS tagging rules, seed aspects similarity thresholds, and sample mapping outcomes.
4. **Dependency-Guided Sentiment Analysis:** Detailed tree-traversal logic (e.g. `amod`, copula structures), negation checking rules, and user/item profile distributions.
5. **Hybrid Recommendation Formulation:** Equations for SVD matrix factorization, aspect weighted alignment, global score normalization, and blending.
6. **CPU Optimization & Parallelization:** Explain how multiprocessing and batching in `spaCy` enabled processing under CPU constraints.
7. **Experimental Setup:** Hyperparameters (α, SVD latent dimensions k, learning

rate, regularization, confidence threshold τ).

8. **Results & Comparison:** Metrics table comparing all 3 baselines against the Hybrid model (RMSE, HR@10, NDCG@10).
9. **Bonus Tasks – Word2Vec / Transformer Extensions (+15% / +30%):** Document Word2Vec/Transformer designs, implementation setup, computational efficiency, and comparative evaluations against mandatory VADER (RMSE, HR@10, NDCG@10).
10. **Ablation Study:** At least 3 controlled experiments (α sweep from 0.0 to 1.0; Rating SVD alone vs. Hybrid; Mandatory VADER vs. Transformer-based sentiment engine).
11. **Conceptual & Mathematical Answers:** Complete, detailed answers to the 3 questions in Section 4.
12. **Error Analysis:** Examine 5 concrete failure cases (≥ 2 aspect noise/sarcasm, ≥ 2 rating/aspect conflict, ≥ 1 cold-start user).
13. **AI Usage Disclosure:** Transparent description of AI tools used during development.

8. Grading

Table 2 – *Grading rubric*

Component	Weight	What is Evaluated
Hidden test recommendation quality	25%	Ranking metrics (HR@10, NDCG@10) on test users
Dev-set quality & Phase 0 data pipeline	15%	Phase 0 preprocessing correctness, metric correctness on dev set + clean execution
Aspect extraction & NLP pipeline (VADER)	25%	Accuracy of POS extraction, semantic mapping, and mandatory dependency-guided VADER
Ablation, Deep Dives & Error analysis	20%	Depth of report sections 10, 11, 12 (α sweep, mathematical answers, failure analysis)
`trace.jsonl` quality	5%	Completeness, correctness, and interpretability of user logs
Code clarity & format compliance	10%	Code modularity, documentation, and strict CLI compliance
Total Mandatory	**100%**	
Bonus: Word2Vec Semantic Extension (Sec 3.7)	**+15% (Bonus)**	Training quality of custom Word2Vec, corpus tokenization, and comparative evaluation
Bonus: Transformer NLP Extension (Sec 3.8)	**+30% (Bonus)**	Implementation quality, VADER vs. Transformer comparative evaluation, and empirical analysis
Maximum Achievable Score	**145%**	

9. Submission

★ Important Note

Submit a single zip archive named:

`NLP-HW3-FirstName_LastName-StudentNumber.zip`

The archive must contain all Python source files (and optional Jupyter Notebooks `.ipynb` used for Colab / Hugging Face bonus experiments), the `results/` folder with output files, and `report.pdf`. Do not include raw Yelp dataset CSV files in the zip submission.

10. Key Hints

- **Use `spacy.pipe` with multi-processing:** Single-threaded `nlp(text)` loops on 100,000 reviews will take hours. Use batching (`batch_size=1000`) and `n_process=4` to finish in minutes.
- **Normalize scores properly:** SVD outputs 1–5 star estimates, while aspect score outputs $[-1, +1]$. Normalize both to $[0, 1]$ globally before applying $\alpha \cdot S_1 + (1 - \alpha) \cdot S_2$.
- **Sparse matrices are essential:** Use `scipy.sparse.csr_matrix` for building User-Item and User-Aspect matrices to keep memory usage under 2 GB.
- **Filter rare aspects:** Restrict your aspect mapping strictly to the 5 canonical categories to maintain robust profiles.

11. Suggested References

1. M. Hu and B. Liu, "Mining and summarizing customer reviews," in *Proceedings of the tenth ACM SIGKDD international conference on Knowledge discovery and data mining*, 2004, pp. 168–177.
2. Y. Koren, R. Bell, and C. Volinsky, "Matrix factorization techniques for recommender systems," *Computer*, vol. 42, no. 8, pp. 30–37, 2009.
3. J. McAuley and J. Leskovec, "Hidden factors and hidden topics: understanding rating dimensions with review text," in *Proceedings of the 7th ACM conference on Recommender systems*, 2013, pp. 165–172.
4. C.J. Hutto and E. Gilbert, "VADER: A parsimonious rule-based model for sentiment analysis of social media text," in *Proceedings of the International AAAI Conference on Web and Social Media*, vol. 8, no. 1, 2014.